

C Programming

Variables and Data Types Ch.4

Ch. 4

Variables and Data Types

Variables and Constants

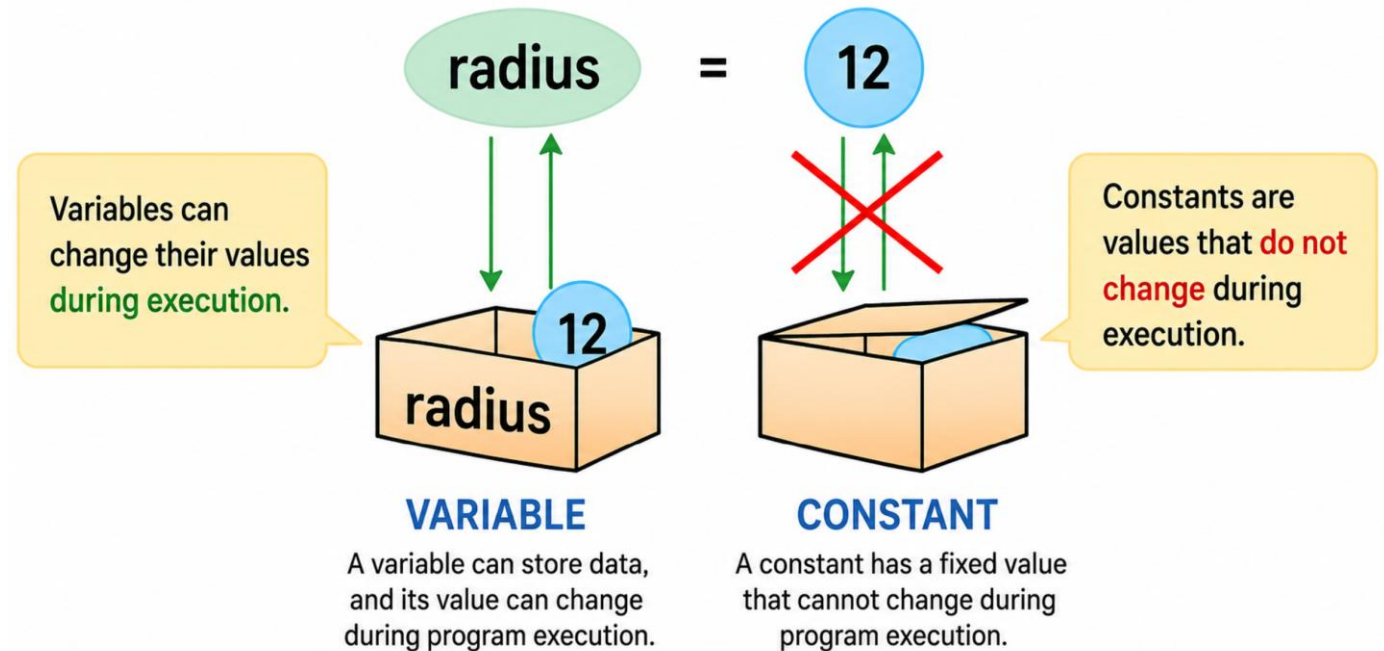
- **Variable**: a space where the stored value can be changed.
- **Constant**: a space where the stored value cannot be changed (example: 3.14, 100, 'A', "Hello World!").
- Variables are created directly in main memory - we use memory space by using **variable** names.

```
/* Program to calculate the area of a circle */
#include <stdio.h>
int main( void )
{
    float radius; // Radius of the circle
    float area;   // Area of a circle

    printf ( "Enter the radius of the circle:" );
    scanf ( "%f" , &radius);

    area = 3.141592 * radius * radius;
    printf ( "Area of the circle: %f \n" , area);

    return 0;
}
```



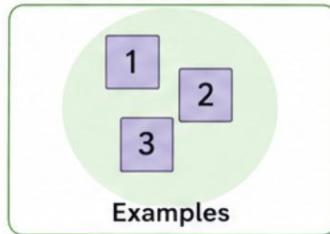
Data Type

A data type specifies the **type of data** a variable can store.
It determines the **size of memory** allocated and the **range of values** that can be stored.

DATA TYPE

INTEGER TYPE

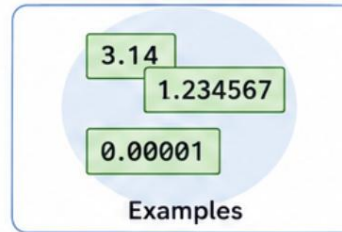
Stores whole numbers
(no decimal point).



Type	Size (bytes)	Range
short	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647
long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
long long	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

FLOATING POINT TYPE

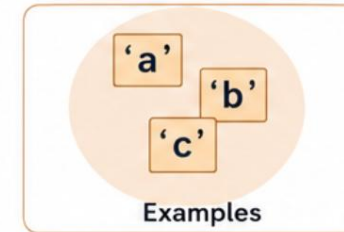
Stores real numbers
(with decimal point).



Type	Size (bytes)	Precision	Range
float	4	~ 6-7 decimal digits	$\pm 1.2\text{E}-38$ to $\pm 3.4\text{E}+38$
double	8	~ 15-16 decimal digits	$\pm 2.2\text{E}-308$ to $\pm 1.7\text{E}+308$
long double	16	~ 18-19 decimal digits	$\pm 3.4\text{E}-4932$ to $\pm 1.1\text{E}+4932$

CHARACTER TYPE

Stores characters such as
letters, digits, symbols.

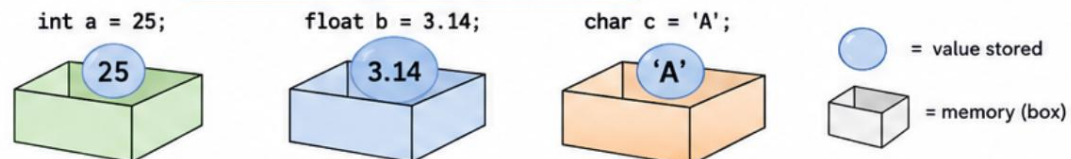


Type	Size (bytes)	Description
char	1	Stores a single character.

KEY POINTS

- Choose the right data type to save memory and improve performance.
- Integer types store whole numbers, floating point types store decimal numbers, and character type stores a single character.

MEMORY VISUALIZATION (EXAMPLE)



The Size of a Data Type

- To find out the size of a data type, use “sizeof” You can use the operator. “sizeof” is an operator that returns the size of a variable or data type in bytes.

Syntax

sizeof()

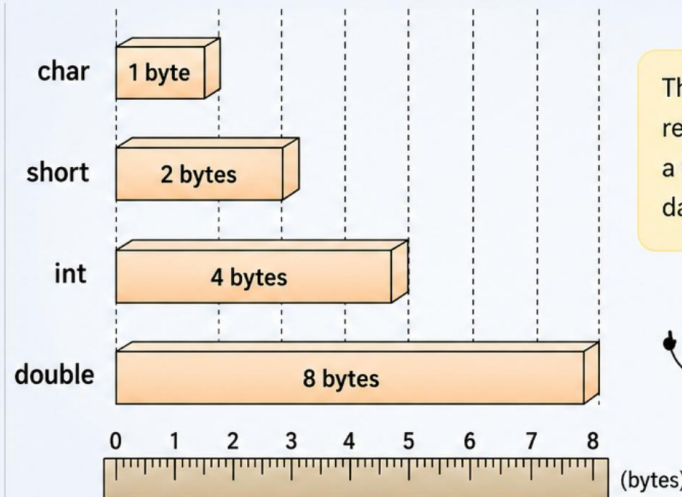
yes

`sizeof(x)` // variable

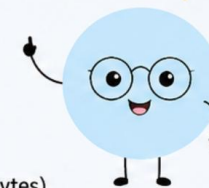
`sizeof(10)` // value

`sizeof(int)` // data type

`sizeof(double)` // data type



The **sizeof** operator returns the size of a **variable** or data type in **bytes**.



- **sizeof()** is a **unary operator**.
- It is used to find out how many **bytes** are required to store a variable or a data type.
- The result of **sizeof()** is of type **size_t** (an unsigned integer type).

sizeof: Example

```
#include <stdio.h>

int main( void )
{
    int x;
    printf ( "Size of variable x: %d\n" , sizeof (x));
    printf ( "Size of char type: %d\n" , sizeof ( char ));
    printf ( "Size of int type: %d\n" , sizeof ( int ));
    printf ( "Size of short type: %d\n" , sizeof ( short ));
    printf ( "Size of long type: %d\n" , sizeof ( long ));
    printf ( "Size of float type: %d\n" , sizeof ( float ));
    printf ( "Size of double type: %d\n" , sizeof ( double ));
    return 0;
}
```

sizeof is an operator that returns the size of a variable or data type in bytes.

Data Type

- Why are there so many different types of integers in C?
To allow programmers to select and use them according to their intended use.
- The number of bits can expand the range of integers, but requires more memory space.

→ To allow programmers to select and use them according to their intended use.



short

Small values
(e.g., counter)



int

General purpose
(e.g., number of items)



long

Larger values
(e.g., population)



long long

Very large values
(e.g., distance in space)

→ Different integer types let you use the right amount of memory for the job.

→ The number of bits can expand the range of integers, but requires more memory space.

Type	short	int	long	long long
Bits	16 bit	32 bit	32 bit	64 bit
Range (signed)	-32,768 ~ 32,767	-2,147,483,648 ~ 2,147,483,647	-2,147,483,648 ~ 2,147,483,647	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
Memory	 2 bytes	 4 bytes	 4 bytes	 8 bytes
	Smaller range Less memory usage			Larger range More memory usage

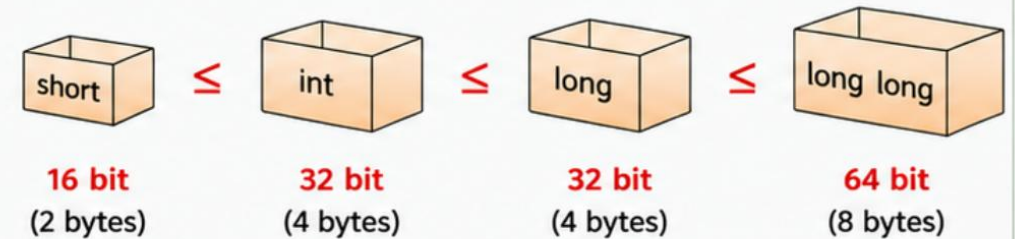
Integer Type

Integer data types store **whole numbers** (no decimal point).

The type you choose determines how much memory is used and the range of values that can be stored.

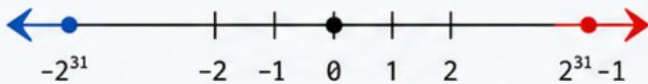
	Data Type	Size (bits)	Size (bytes)	Range
Integer Type	short	16 bit	2 bytes	-32768 ~ 32767
	int	32 bit	4 bytes	-2147483648 ~ 2147483647
	long	32 bit	4 bytes	-2147483648 ~ 2147483647
	long long	64 bit	8 bytes	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807

MEMORY SIZE COMPARISON



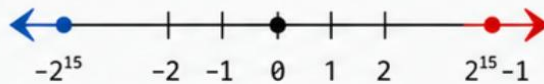
RANGE OF int (32-bit)

$-2^{31}, \dots, -2, -1, 0, 1, 2, \dots, 2^{31}-1$
(-2147483648 ~ +2147483647)



RANGE OF short (16-bit)

$-2^{15}, \dots, -2, -1, 0, 1, 2, \dots, 2^{15}-1$
(-32768 ~ +32767)



MIN / MAX VALUES IN BINARY

int (32-bit)

Min value: $-2^{31} = -2147483648$

1 0
(31 zeros)

Max value: $2^{31}-1 = 2147483647$

0 1
(31 ones)

short (16-bit)

Min value: $-2^{15} = -32768$

1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(15 zeros)

Max value: $2^{15}-1 = 32767$

0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
(15 ones)



- Choose the appropriate data type based on the range of values your program needs and memory efficiency.
- long long** provides the largest range among the basic integer types.

Representing 100 in Integer Type

100 (decimal) = 1100100 (binary)

char
(8-bit)

sign bit

value bits (7-bit)

0	0	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---	---

Decimal : 100

Binary : 01100100

Range

-128 ~ 127

short
(16-bit)

sign bit

value bits (15-bit)

0	0	0	0	0	0	0	1	1	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Decimal : 100

Binary : 0000000001100100

Range

$-32,768 \sim 32,767$

int
(32-bit)

sign bit

value bits (31-bit)

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Decimal : 100

Binary : 0000000000000000000000001100100

Range

-2,147,483,648

~ 2,147,483,647

long
(64-bit)

sign bit

value bits (63-bit)

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 1 0 0

Decimal : 100

Binary : 00000000000000000000000000000000
000000000000000001100100

Range

-9,223,372,036,854,775,808

~ 9,223,372,036,854,775,807

long long
(64-bit)

sign bit

value bits (63-bit)

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Decimal : 100

Binary : 000000000000000000000000(0000000
0000000000000001100100

Range

-9,223,372,036,854,775,808

~ 9,223,372,036,854,775,807

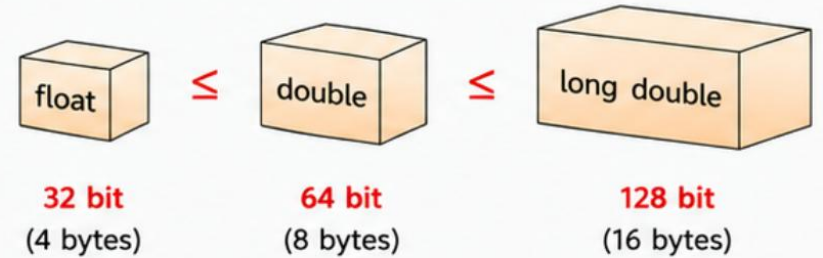
Floating-Point Type

Floating point data types store **real numbers** (with decimal point).

The type you choose determines how much memory is used, the precision (number of decimal digits), and the range of values that can be stored.

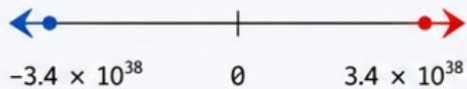
	Data Type	Size (bits)	Size (bytes)	Precision (approx.)	Range
Floating Point Type	float	32 bit	4 bytes	~ 6 – 7 decimal digits	$\pm 1.2\text{E-}38$ to $\pm 3.4\text{E+}38$
	double	64 bit	8 bytes	~ 15 – 16 decimal digits	$\pm 2.2\text{E-}308$ to $\pm 1.7\text{E+}308$
	long double	128 bit	16 bytes	~ 18 – 19 decimal digits	$\pm 3.4\text{E-}4932$ to $\pm 1.1\text{E+}4932$

MEMORY SIZE COMPARISON



RANGE OF float (32-bit)

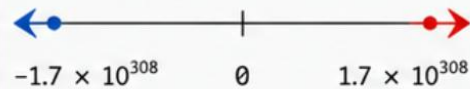
$$\pm 1.2 \times 10^{-38} \sim \pm 3.4 \times 10^{38}$$



Very small numbers (near 0):
 $\pm 1.2 \times 10^{-38}$

RANGE OF double (64-bit)

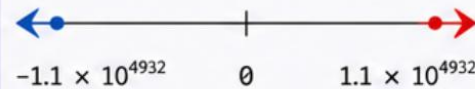
$$\pm 2.2 \times 10^{-308} \sim \pm 1.7 \times 10^{308}$$



Very small numbers (near 0):
 $\pm 2.2 \times 10^{-308}$

RANGE OF long double (128-bit)

$$\pm 3.4 \times 10^{-4932} \sim \pm 1.1 \times 10^{4932}$$



Very small numbers (near 0):
 $\pm 3.4 \times 10^{-4932}$

PRECISION (NUMBER OF DECIMAL DIGITS)

float (~6–7 digits)

Example: 3.1415926

double (~15–16 digits)

Example: 3.141592653589793

long double (~18–19 digits)

Example: 3.141592653589793238



- Use **float** when memory is limited and 6–7 digits precision is enough.
- Use **double** for general purpose (better precision and range).

- Use **long double** when you need very high precision and a larger range.

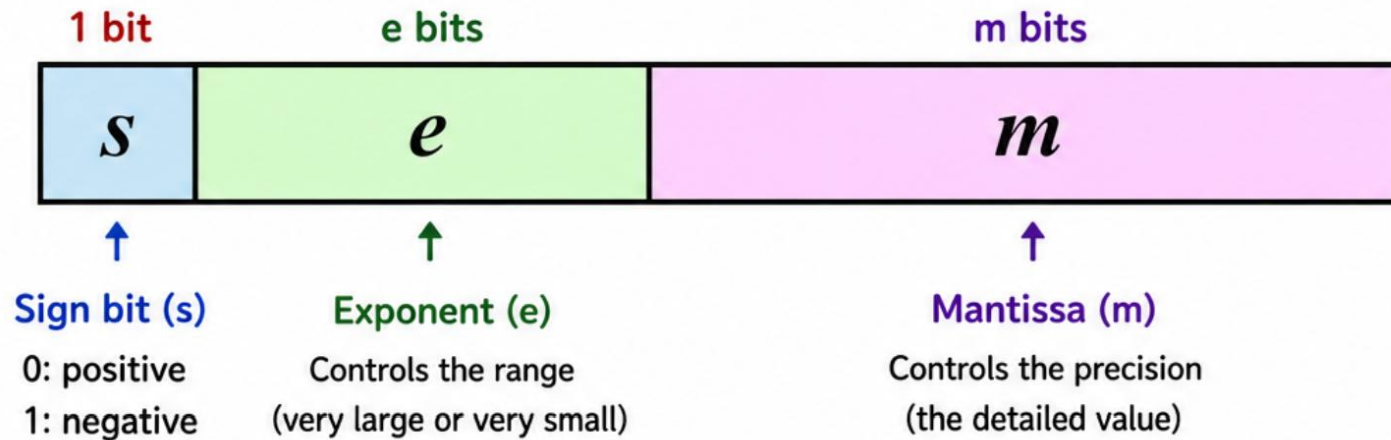
Floating-Point Type representation method



What is the largest number that can be represented this way?

It depends on the number of bits. Larger exponent (e) and mantissa (m) allow representing larger numbers!

A floating-point number is stored like this:



Value is calculated as:

$$(-1)^s \times 1.m \times 2^{(e - \text{bias})}$$

(bias is a fixed value)



There have been a number of different floating point methods in use for some time, but they have been standardized since 1985 as **IEEE 754**.

Common IEEE 754 formats

- Single precision (32-bit) : 1-8-23 bits (s-e-m)
- Double precision (64-bit) : 1-11-52 bits (s-e-m)

Representing 3.14 in Floating-Point Type

- Why 3.14 becomes 1.100...?

In binary, $3.14 = 11.00100011110101110000101\dots_2$.

To store it in normalized form ($1.x \times 2^e$), we move the binary point left one place: $11.001000\dots_2 = 1.100100011110101110000101\dots_2 \times 2^1$.

float (32-bit)

1 sign bit
8 exponent bits
23 fraction bits

sign (1 bit)

0

$s = 0$
(positive)

exponent (8 bits)

1 0 0 0 0 0 0 0

Actual exponent = 1

Bias = 127

Stored exponent = $1 + 127 = 128$

= 10000000_2

fraction / mantissa (23 bits)

1 0 0 1 0 0 0 1 1 1 1 0 0 1 1 1 0 0 0 0 0 0 1 0

Mantissa

= $1.10010001111010111000010_2 - 1$

= 10010001111010111000010 (23 bits)

double (64-bit)

1 sign bit
11 exponent bits
52 fraction bits

sign (1 bit)

0

$s = 0$
(positive)

exponent (11 bits)

1 0 0 0 0 0 0 0 0 0 0

Actual exponent = 1

Bias = 1023

Stored exponent = $1 + 1023 = 1024$

= 1000000000_2

fraction / mantissa (52 bits)

1 0 0 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 0 0 0 1 0 0 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 0 0 0 0

Mantissa

= $1.10010001111010111000010011000010111010111000000000000000000000_2 - 1$

= $10010001111010111000010011000010111010111000000000000000000000$ (52 bits)

long double (80-bit)

1 sign bit
15 exponent bits
64 fraction bits
(extended precision)

sign (1 bit)

0

$s = 0$
(positive)

exponent (15 bits)

1 0 0 0 0 0 0 0 0 0 0 0 0 0 0

Actual exponent = 1

Bias = 16383

Stored exponent = $1 + 16383 = 16384$

= 010000000000000_2

fraction / mantissa (64 bits)

1 0 0 1 0 0 0 1 1 1 0 1 1 1 1 1 1 1 0 0 0 1 0 0 1 0 0 0 0 0 1 1 1 1 1 0 0 1 1 0 0 0 1 0 0 0 0 0 0

Mantissa (explicit integer bit + fractional part)

= $1.100100011110101110000100110000100111010111000$

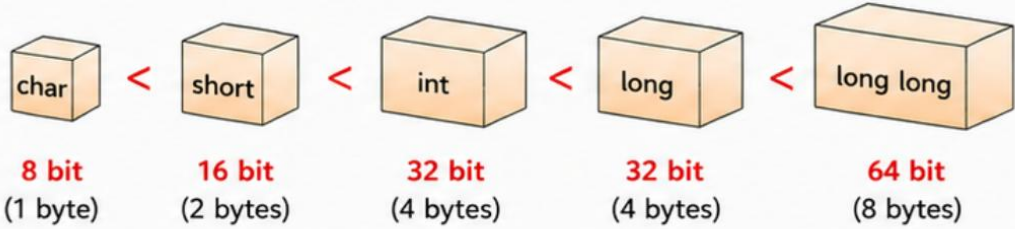
(64 bits)

Character Type: Number

Character data type stores a **single character** such as letters, digits, symbols, or special characters. It is used to store text and individual characters in a program.

	Data Type	Size (bits)	Size (bytes)	Range / Values	Description
Character Type	char	8 bit	1 byte	-128 ~ 127 (or) 0 ~ 255 (implementation dependent)	Stores a single character. Uses ASCII or extended ASCII encoding.

MEMORY SIZE COMPARISON



CHARACTER EXAMPLES

Printable Characters

'A' 'b' '9' '\$' '@'

Stored as their ASCII
(or extended ASCII) code.

Escape Sequences (non-printable)

'\n' '\t' '\0' '\\' '\'

Represent special characters
like newline, tab, null, etc.

ASCII TABLE (EXAMPLES)

Character	ASCII Code (Decimal)	ASCII Code (Binary)	Hex
'A'	65	01000001	0x41
'Z'	90	01011010	0x5A
'a'	97	01100001	0x61
'z'	122	01111010	0x7A
'0'	48	00110000	0x30
'9'	57	00111001	0x39
'\n' (newline)	10	00001010	0x0A
'\0' (null)	0	00000000	0x00

HOW char STORES A CHARACTER

Example: char ch = 'A';

'A' has ASCII code 65.

Memory (1 byte)

0 1 0 0 0 0 0 1

↑
bit 7 (MSB)

↑
bit 0 (LSB)

Binary: 01000001
Decimal: 65 Hex: 0x41

Example: char ch = '\n';

'\n' has ASCII code 10.

Memory (1 byte)

0 0 0 0 1 0 1 0

↑
bit 0 (LSB)

Binary: 00001010
Decimal: 10 Hex: 0x0A

COMMON USES OF char

- Storing single characters
- Processing text and strings
- Reading characters from keyboard
- Representing symbols and special characters

IMPORTANT NOTES

- **char** is always 1 byte (8 bits) in C (on most systems).
- Signed char range: **-128 to 127** | Unsigned char range: **0 to 255**
- The actual range may depend on the compiler and system.

SUMMARY



char is used to store one character using 1 byte (8 bits) of memory. It is the basic building block for handling text and characters in C.

Character Type: Text

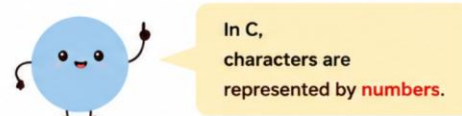
- Text is more important to humans than to computers. Letters are also expressed using numbers.
- ASCII code (American Standard Code for Information Interchange).
- char type: used to store text. char code; code = 'A';

In C, **characters** are represented by **numbers** using the **ASCII** (American Standard Code for Information Interchange). There is a **one-to-one correspondence** between each character and its ASCII code.

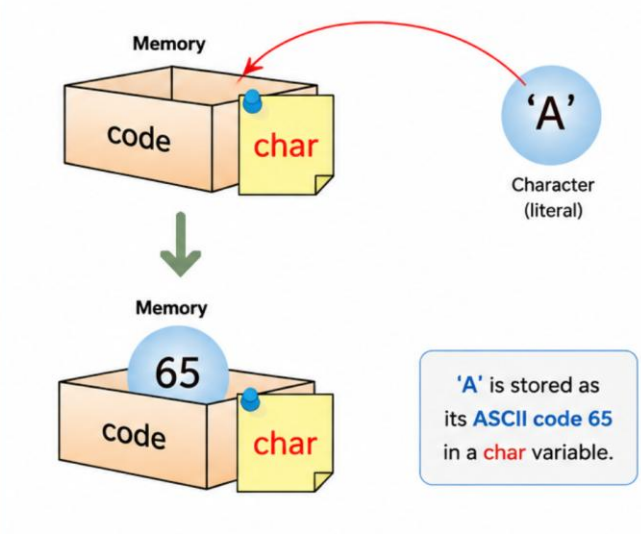
1. Characters are represented by numbers (ASCII codes)



Character	A	E	G	J	N
ASCII Code (Decimal)	65	69	71	74	78



2. How a character is stored in memory (char type)



'A' is stored as its **ASCII code 65** in a **char** variable.

Summary

- Each character has a unique **ASCII code** (0 ~ 127).
- In C, **char** variables store these codes as numbers.
- This allows characters to be processed like numbers in programs.

Examples

Character	'A'	'B'	'a'	'0'	'\n'
ASCII Code (Decimal)	65	66	97	48	10



Characters look like letters, but in C, they are **numbers**!

Number Representation: Decimal and Hexadecimal



Computers use only 0 and 1 (binary digits), but we use decimal (base 10) and hexadecimal (base 16) to make numbers shorter and easier to read.

1 Decimal (Base 10)

- Uses 10 digits: 0–9
- Example: 18 (decimal)

Digits used

0	1	2	3	4
5	6	7	8	9

2 Hexadecimal (Base 16)

- Uses 16 digits: 0–9 and A–F
- Example: 0x12 (hexadecimal) = 18 (decimal)

Digits used

0	1	2	3	4	5	6	7
8	9	A	B	C	D	E	F

Conversion Example (18)

Decimal

18

÷ 16

←

× 16

Hexadecimal

0x12

How it works

$18 \div 16 = 1 \dots 2$
(remainder 2)

$1 \div 16 = 0 \dots 1$
(remainder 1)

Read in reverse → 0x12

Decimal and Hexadecimal Comparison (0~18)

Decimal (Base 10)	Hexadecimal (Base 16)
0	0x0
1	0x1
2	0x2
3	0x3
4	0x4
5	0x5
6	0x6
7	0x7
8	0x8
9	0x9
10	0xA
11	0xB
12	0xC
13	0xD
14	0xE
15	0xF
16	0x10
17	0x11
18	0x12



There are many ways to represent the same value.
Computers internally store everything as 0s and 1s.



TIP!

Hexadecimal is a compact way to represent binary, using 4 bits per digit!

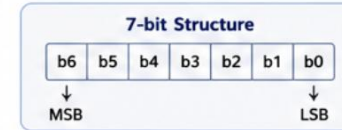
ASCII Code Table (One character)

ASCII uses 7 bits (128 codes)

- 0 ~ 127
- 32 ~ 126 : printable characters

ASCII Code Table

ASCII (American Standard Code for Information Interchange) is a character encoding standard that represents text in computers and communication equipment.



Dec	Hex	Binary (7-bit)	Char	Description
0	0x00	0000000	NUL	Null character
1	0x01	0000001	SOH	Start of Heading
2	0x02	0000010	STX	Start of Text
3	0x03	0000011	ETX	End of Text
4	0x04	0000100	EOT	End of Transmission
5	0x05	0000101	ENQ	Enquiry
6	0x06	0000110	ACK	Acknowledge
7	0x07	0000111	BEL	Bell
8	0x08	0001000	BS	Backspace
9	0x09	0001001	TAB	Horizontal Tab
10	0x0A	0001010	LF	Line Feed
11	0x0B	0001011	VT	Vertical Tab
12	0x0C	0001100	FF	Form Feed
13	0x0D	0001101	CR	Carriage Return
14	0x0E	0001110	SO	Shift Out
15	0x0F	0001111	SI	Shift In
16	0x10	0010000	DLE	Data Link Escape
17	0x11	0010001	DC1	Device Control 1
18	0x12	0010010	DC2	Device Control 2
19	0x13	0010011	DC3	Device Control 3
20	0x14	0010100	DC4	Device Control 4
21	0x15	0010101	NAK	Negative Acknowledge
22	0x16	0010110	SYN	Synchronous Idle
23	0x17	0010111	ETB	End of Transmission Block
24	0x18	0011000	CAN	Cancel
25	0x19	0011001	EM	End of Medium
26	0x1A	0011010	SUB	Substitute
27	0x1B	0011011	ESC	Escape
28	0x1C	0011100	FS	File Separator
29	0x1D	0011101	GS	Group Separator
30	0x1E	0011110	RS	Record Separator
31	0x1F	0011111	US	Unit Separator

Dec	Hex	Binary (7-bit)	Char	Description
32	0x20	0100000	(space)	Space
33	0x21	0100001	!	Exclamation mark
34	0x22	0100010	"	Double quote
35	0x23	0100011	#	Number sign
36	0x24	0100100	\$	Dollar sign
37	0x25	0100101	%	Percent sign
38	0x26	0100110	&	Ampersand
39	0x27	0100111	'	Apostrophe
40	0x28	0101000	(	Left parenthesis
41	0x29	0101001	)	Right parenthesis
42	0x2A	0101010	*	Asterisk
43	0x2B	0101011	+	Plus sign
44	0x2C	0101100	,	Comma
45	0x2D	0101101	-	Hyphen (minus)
46	0x2E	0101110	.	Period
47	0x2F	0101111	/	Slash
48	0x30	0110000	0	Digit zero
49	0x31	0110001	1	Digit one
50	0x32	0110010	2	Digit two
51	0x33	0110011	3	Digit three
52	0x34	0110100	4	Digit four
53	0x35	0110101	5	Digit five
54	0x36	0110110	6	Digit six
55	0x37	0110111	7	Digit seven
56	0x38	0111000	8	Digit eight
57	0x39	0111001	9	Digit nine
58	0x3A	0111010	:	Colon
59	0x3B	0111011	;	Semicolon
60	0x3C	0111100	<	Less than
61	0x3D	0111101	=	Equal sign
62	0x3E	0111110	>	Greater than
63	0x3F	0111111	?	Question mark

Dec	Hex	Binary (7-bit)	Char	Description
64	0x40	1000000	@	At sign
65	0x41	1000001	A	Uppercase A
66	0x42	1000010	B	Uppercase B
67	0x43	1000011	C	Uppercase C
68	0x44	1000100	D	Uppercase D
69	0x45	1000101	E	Uppercase E
70	0x46	1000110	F	Uppercase F
71	0x47	1000111	G	Uppercase G
72	0x48	1001000	H	Uppercase H
73	0x49	1001001	I	Uppercase I
74	0x4A	1001010	J	Uppercase J
75	0x4B	1001011	K	Uppercase K
76	0x4C	1001100	L	Uppercase L
77	0x4D	1001101	M	Uppercase M
78	0x4E	1001110	N	Uppercase N
79	0x4F	1001111	O	Uppercase O
80	0x50	1010000	P	Uppercase P
81	0x51	1010001	Q	Uppercase Q
82	0x52	1010010	R	Uppercase R
83	0x53	1010011	S	Uppercase S
84	0x54	1010100	T	Uppercase T
85	0x55	1010101	U	Uppercase U
86	0x56	1010110	V	Uppercase V
87	0x57	1010111	W	Uppercase W
88	0x58	1011000	X	Uppercase X
89	0x59	1011001	Y	Uppercase Y
90	0x5A	1011010	Z	Uppercase Z
91	0x5B	1011011	[	Left bracket
92	0x5C	1011100	\	Backslash
93	0x5D	1011101	]	Right bracket
94	0x5E	1011110	^	Caret
95	0x5F	1011111	_	Underscore

Dec	Hex	Binary (7-bit)	Char	Description
96	0x60	1100000	`	Grave accent
97	0x61	1100001	a	Lowercase a
98	0x62	1100010	b	Lowercase b
99	0x63	1100011	c	Lowercase c
100	0x64	1100100	d	Lowercase d
101	0x65	1100101	e	Lowercase e
102	0x66	1100110	f	Lowercase f
103	0x67	1100111	g	Lowercase g
104	0x68	1101000	h	Lowercase h
105	0x69	1101001	i	Lowercase i
106	0x6A	1101010	j	Lowercase j
107	0x6B	1101011	k	Lowercase k
108	0x6C	1101100	l	Lowercase l
109	0x6D	1101101	m	Lowercase m
110	0x6E	1101110	n	Lowercase n
111	0x6F	1101111	o	Lowercase o
112	0x70	1110000	p	Lowercase p
113	0x71	1110001	q	Lowercase q
114	0x72	1110010	r	Lowercase r
115	0x73	1110011	s	Lowercase s
116	0x74	1110100	t	Lowercase t
117	0x75	1110101	u	Lowercase u
118	0x76	1110110	v	Lowercase v
119	0x77	1110111	w	Lowercase w
120	0x78	1111000	x	Lowercase x
121	0x79	1111001	y	Lowercase y
122	0x7A	1111010	z	Lowercase z
123	0x7B	1111011	{	Left brace
124	0x7C	1111100		Vertical bar
125	0x7D	1111101	}	Right brace
126	0x7E	1111110	~	Tilde
127	0x7F	1111111	DEL	Delete

Summary



- **Total:** 128 characters (0 ~ 127)
- 7-bit binary representation

Control characters: 0 ~ 31, 127
(used for communication control)

Printable characters: 32 ~ 126
(space, numbers, letters, symbols)



- Widely used in text processing, communication, and file formats.

How to represent Other Languages: Unicode

Unicode is a universal character encoding standard that can represent all characters in all languages consistently. Each character has a **unique code point**, and Unicode uses **16 bits or more** to represent it.

1. What is Unicode?

- Unicode assigns a unique number (code point) to every character in every language.
- It supports over 11,172 Hangul syllables, symbols, emojis, and more.
- Unicode uses 16 bits (or more) to represent characters.

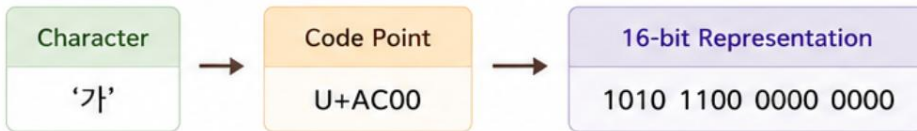
Character	Unicode Code Point (Hex)	Binary (16-bit)	Decimal
A	U+0041	0000 0000 0100 0001	65
가	U+AC00	1010 1100 0000 0000	44,032
한	U+D55C	1101 0101 0101 1100	54,140
😊	U+1F642	0001 1111 0110 0100 0010	128,578
🌍	U+1F30D	0001 1111 0011 0000 1101	127,757



Each character has its own unique code point.
Example: '가' = U+AC00

2. How Unicode Represents a Character

- Every character is represented by its code point.
- Unicode uses 16 bits (2 bytes) or more to store the code point.



What is a Code Point?

A code point is a unique number assigned to each character in Unicode. It is written in the form U+XXXX (hexadecimal).



Unicode can use more than 16 bits when necessary (e.g., emojis), but most common characters (including Hangul) are within 16 bits.

3. Unicode Can Express All Characters Consistently

- Supports all languages, symbols, and emojis.
- Same character = same code point everywhere.
- Enables consistent text processing across different systems and platforms.
- Managed by the Unicode Consortium.



Unicode.
Consortium

Maintains the Unicode Standard
Latest version: Unicode 15.1 (as of 2024)

Examples from Different Languages

A	1	가	中	😊	🌍
U+0041	U+0031	U+AC00	U+4E2D	U+1F642	U+1F30D



Unicode is the global standard that makes it possible to represent and process **any text, in any language, on any platform**.

Character Variable and char as Integer

```
char code1 = 'A' ; // Initialize to a character constant
char code2 = 65;  // Initialize to ASCII code

char code = 'A';
printf ( "%d %d %d \n" , code, code + 1, code + 2); // 65 66 67
printf ( "%c %c %c \n" , code, code + 1, code + 2); // A B C
```

The char type can be used to store 8-bit integers.

Control Characters and Escape Sequences

- **Control characters**

Control characters are special characters that perform an action instead of displaying a visible character.

Examples: Newline → moves the cursor to the next line
 Tab → moves the cursor to the next tab position
 Alert → produces a beep sound

- **Escape Sequences**

Escape sequences are special notations used in C code to represent control characters or other special characters.

Examples: \n → represents the newline control character
 \t → represents the tab control character
 \a → represents the alert control character
 \" → represents a double quotation mark
 \\ → represents a backslash

Escape Sequences



Control characters are **special characters** that perform functions such as formatting output, sound, moving the cursor, and clearing the screen. In C language, these are expressed using **escape sequences**.

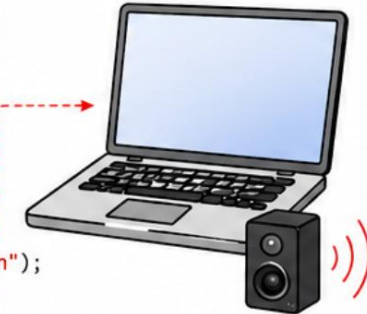
Main Control Characters

Escape Sequence	Name	Meaning	ASCII (Decimal)	Example Output / Effect
<code>\n</code>	Newline (Line Feed)	Moves to the next line	10	Hello World
<code>\t</code>	Tab	Moves to the next tab stop (usually every 4 or 8 spaces)	9	A→→→→B
<code>\r</code>	Carriage Return	Moves the cursor to the beginning of the line	13	ABC\rD DBC
<code>\\</code>	Backslash	Outputs a backslash (\)	92	\
<code>\'</code>	Apostrophe	Outputs a single quote (')	39	'
<code>\"</code>	Quotation Mark	Outputs a double quote (")	34	"
<code>\a</code>	Alert (Bell)	Produces an alert (beep sound)	7	🔊))
<code>\b</code>	Backspace	Moves the cursor one position backward	8	AB\bC AC
<code>\f</code>	Form Feed	Moves to the next page (used for printers)	12	(New Page)
<code>\v</code>	Vertical Tab	Moves to the next vertical tab (rarely used)	11	A : B
<code>\0</code>	Null Character	Indicates the end of a string	0	(End of String)

Example Code and Output

```
#include <stdio.h>

int main(void)
{
    printf("First line.\n");
    printf("Second line with a \t\t\t.\n");
    printf("ABC\rD : Carriage return example\n");
    printf("Beep sound: \a\n");
    printf("Output: \"Hello\" , Small quote: \'A\'\n");
    printf("Backslash output: \\ \n");
    return 0;
}
```



Example Output (On Screen)

First line.
Second line with a tab.
DBC : Carriage return example
Beep sound: 🔊) (beep)
Output: "Hello" , Small quote: 'A'
Backslash output: \ \



Tip

Control characters are not visible on the screen directly, but they control the output format or behavior.

Usage Examples

- Newline: start a new line
- Tab: align columns
- Carriage Return: overwrite from the beginning of the line
- Backspace: delete the previous character
- Alert (Bell): make a beep sound
- Null: end of string

Key Points

- ✓ Escape sequences start with a backslash (\).
- ✓ Their ASCII codes are in the range 0–31 (non-printable characters).
- ✓ Used for screen control, cursor movement, and system control.
- ✓ They improve readability and efficiency of programs.

Quick Reference

Category	Function	Examples
Output Control	Newline, Tab, Carriage Return	<code>\n</code> , <code>\t</code> , <code>\r</code>
Output Formatting	Apostrophe, Quote, Backslash	<code>\'</code> , <code>\"</code> , <code>\\</code>
System Control	Alert (Bell), Form Feed, Vertical Tab	<code>\a</code> , <code>\f</code> , <code>\v</code>
String Control	Null (end of string)	<code>\0</code>

Remember!

Using control characters well makes your program output cleaner and more professional, and improves the user experience.



Integer representation method

1 Binary Numbers and Decimal Values

A binary number is made up of **0s and 1s**.

Each position represents a value.

(Counting starts from 0.)



Binary numbers are used in computers because computers understand only two states: 0 or 1.

bit pattern	decimal value	note
0000000000000000	0	positive integer
0000000000000001	1	
0000000000000010	2	
0000000000000011	3	
0000000000000100	4	
0000000000000101	5	
...	...	

What does it mean?

- Each bit can be 0 or 1.
- With n bits, we can represent 2^n different values.
(For example, 2 bits $\rightarrow 2^2 = 4$ values: 0–3)
- Computers store numbers using these binary patterns.

2 Switches and Binary Values

A switch has two states: OFF (0) or ON (1).

With multiple switches, we can represent different binary numbers.

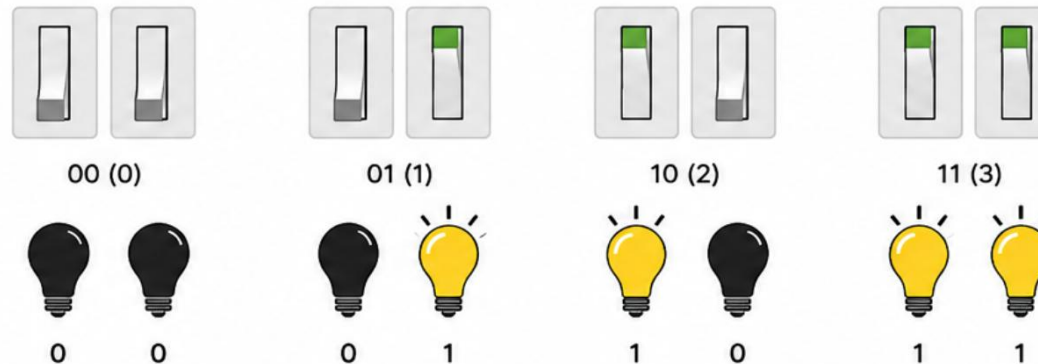
If there is a switch



Example (Using a light)



If there are two switches



2 switches $\rightarrow 2^2 = 4$ possible combinations: 00, 01, 10, 11 (decimal: 0, 1, 2, 3)

Sign bit: Positive vs. Negative Numbers (Signed Integer)

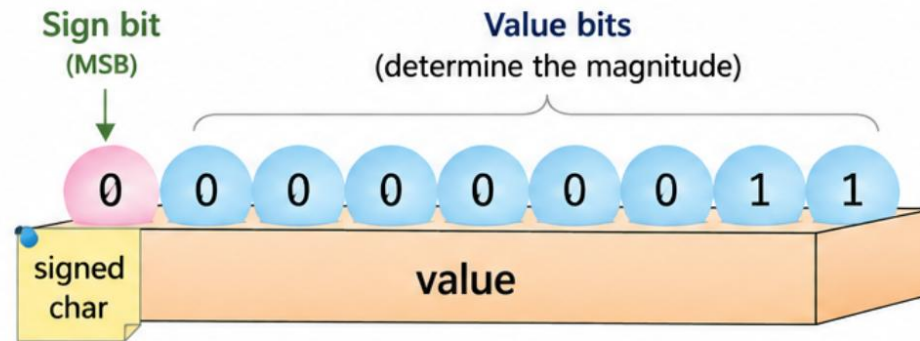


In a signed integer, the leftmost bit (MSB) is the **sign bit**.

- If the sign bit is **0** → the number is non-negative (positive or zero).
- If the sign bit is **1** → the number is negative.

Sign bit = 0 → Positive

If the sign bit is **0**, it is considered a **positive** (or zero) number.

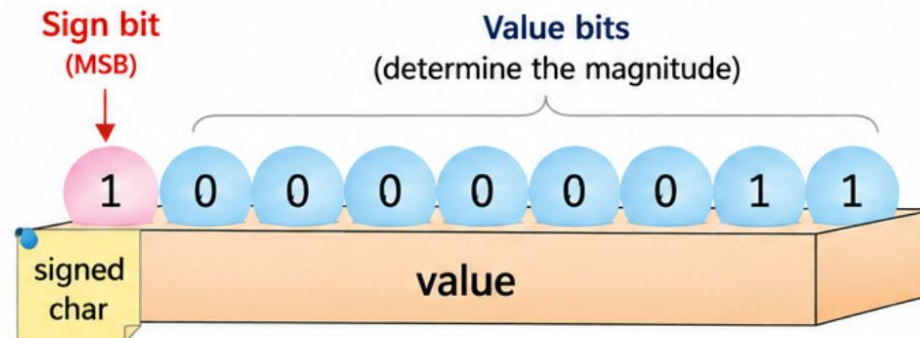


Example (8-bit)

$00000011_2 = 3_{10}$
Sign bit = **0** → Positive

Sign bit = 1 → Negative

If the sign bit is **1**, it is considered a **negative** number.



Example (8-bit)

$10000011_2 = -125_{10}$
Sign bit = **1** → Negative



Note Negative numbers are represented using **two's complement** in most computers.

- 1) Invert all the bits. 2) Add **1**. 3) The result represents the negative value.

Example: +3 to -3 (8-bit)

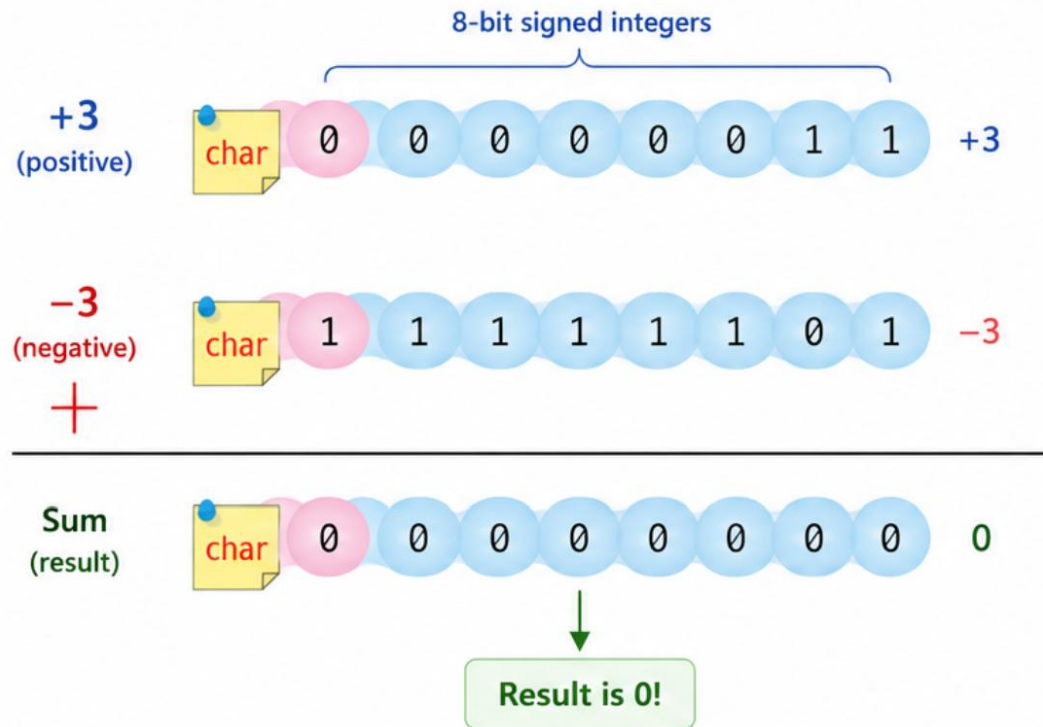
+3: 00000011 → Invert: 11111100 → +1: 11111101 (-3)

Adding Positive and Negative Numbers (Two's Complement)



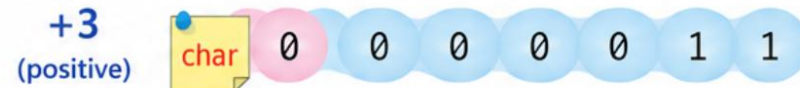
In **two's complement**, negative numbers are represented so that we can add positive and negative numbers simply by adding their bits.

Example: $(+3) + (-3) = 0$ (8-bit)



How -3 is represented (8-bit two's complement)

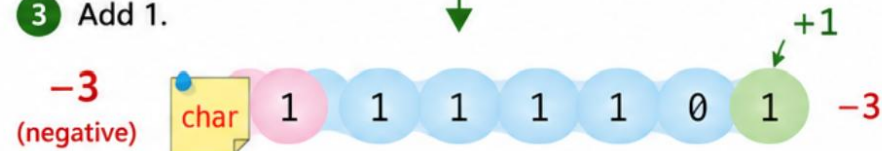
1 Start with +3.



2 Invert all bits ($0 \leftrightarrow 1$).



3 Add 1.



! So, -3 in 8-bit two's complement is **1111101**.



Summary

Using two's complement, we can add and subtract signed integers using the same binary addition.

$$(+3) + (-3) = 0$$

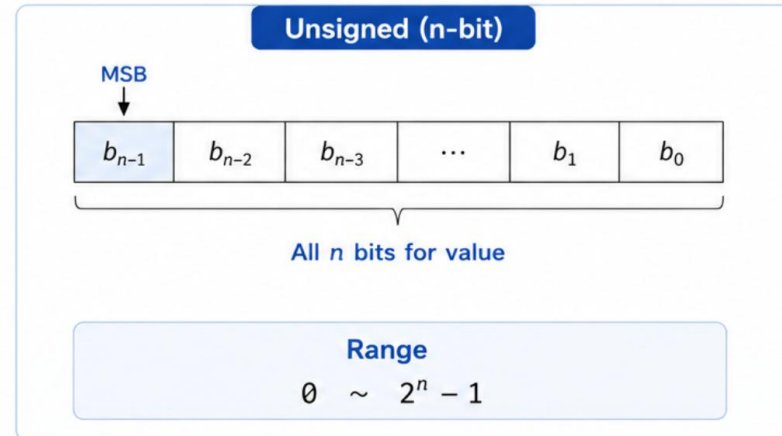
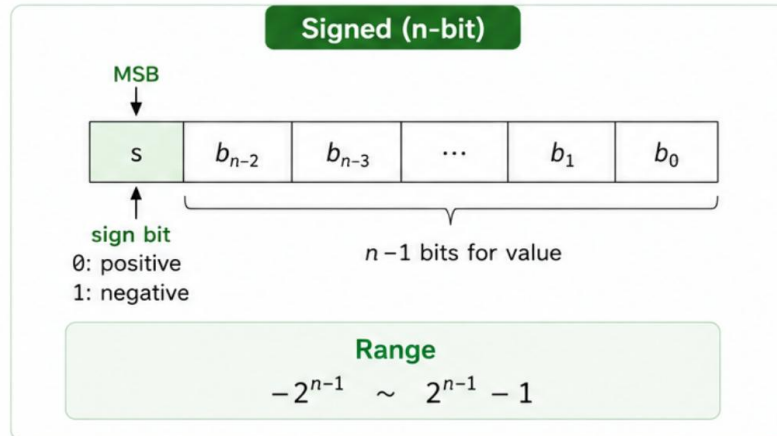
signed, unsigned Modifiers

- **unsigned**: means that only non-negative values are represented (unsigned int).
- **signed**: means that it represents a value with a sign - commonly omitted, since it is the default.

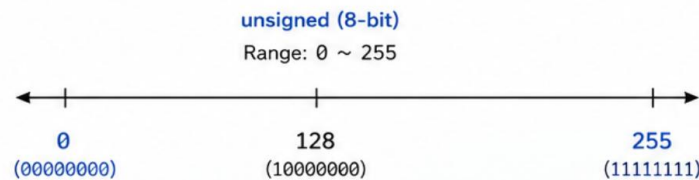
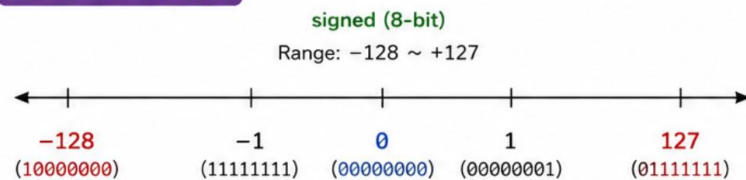
The leftmost bit (MSB) is used differently.

Signed: used as a **sign bit** (0: positive, 1: negative)

Unsigned: used as data (all bits represent the value)



Example (8-bit)



Key Points

- **Signed** integers can represent both negative and positive numbers.
- **Unsigned** integers can represent only non-negative numbers.
- Unsigned provides a wider range for the same number of bits.
- Choose the type based on the values your program needs to store.

Overflow & Underflow in C



- Integers in C have a **limited range**.
- If a calculation goes beyond this range, the value **"wraps around"** and becomes incorrect.



Overflow

When the result is **greater** than the maximum value that the type can store.

Too big → wraps to a small (negative) value



Underflow

When the result is **smaller** than the minimum value that the type can store.

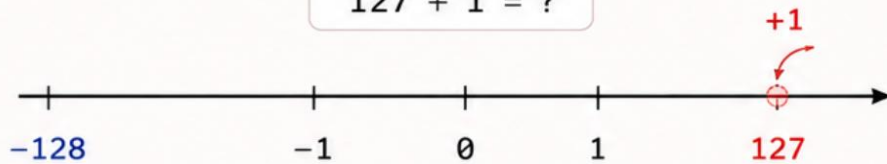
Too small → wraps to a large (positive) value

Examples (8-bit signed int)

Range: -128 ~ +127

Overflow Example

$$127 + 1 = ?$$

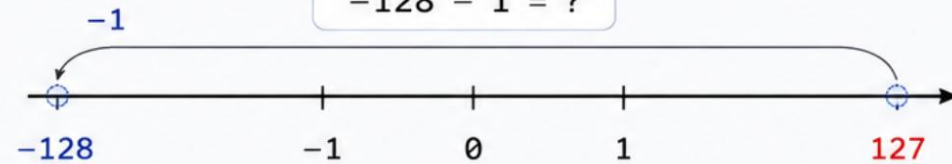


127 is the **maximum**.
Adding 1 goes beyond the range,
so it wraps around to -128.

In C:
 $127 + 1 \rightarrow -128$

Underflow Example

$$-128 - 1 = ?$$



-128 is the **minimum**.
Subtracting 1 goes below the range,
so it wraps around to +127.

In C:
 $-128 - 1 \rightarrow 127$

Key Points

- ✓ **Overflow**: result > maximum → wraps to minimum side
- ✓ **Underflow**: result < minimum → wraps to maximum side
- ✓ **C does not report an error**. It just wraps around silently.



Note

Overflow and underflow can cause bugs in your program. Always be careful with limits of data types!



Example: Overflow & Underflow

```
#include <stdio.h>
#include <limits.h>

int main(void)
{
    unsigned short x = USHRT_MAX;    // 65535
    unsigned short y = 0;

    // Overflow
    printf("Before overflow : %u\n", x);
    x = x + 1;
    printf("After overflow  : %u\n", x);

    // Underflow
    printf("Before underflow: %u\n", y);
    y = y - 1;
    printf("After underflow : %u\n", y);

    return 0;
}
```

Overflow: $65535 + 1 \rightarrow 0$

Underflow: $0 - 1 \rightarrow 65535$

Overflow: occurs when you try to store a number that exceeds the range a variable can represent - it behaves like a car odometer rolling over.

Symbolic Constant

- **Constant** expressed using symbols.

Example: $won = 1120 * dollar$; vs. $won = EXCHANGE_RATE * dollar$;
(using an **actual value**) (using a symbolic **constant**)

- Advantages of symbolic constants: readability is improved, and the values can be easily changed.

Magic Numbers vs. Symbolic Constants in C



Magic numbers are literal values written directly in the code (e.g., 1020, 1050).

Symbolic constants (macros or **const** variables) give a **meaningful name** to those values.

→ Improves readability, maintainability, and reduces errors.

1. Using Literal Constants (Magic Numbers)

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    ...  
    won1 = 1020 * dollar1;
```

```
    won2 = 1020 * dollar2;
```

```
    ...  
}
```

1020

1020



If the exchange rate changes (e.g., 1020 → 1050),
you must change every place where 1020 appears in the code.

Hard to find all occurrences → **Higher chance of errors**

2. Using Symbolic Constants

```
#include <stdio.h>
```

```
#define EXCHANGE_RATE 1020
```

1020

```
int main(void)
```

```
{
```

```
    ...
```

```
    won1 = EXCHANGE_RATE * dollar1;
```

```
    won2 = EXCHANGE_RATE * dollar2;
```

```
    ...  
}
```



If the exchange rate changes,
you only need to change it in **one place** (the definition).

Easy to maintain → **Fewer errors**

Advantages of Symbolic Constants

- ✓ Improves code **readability** (meaningful names)
- ✓ Easy to update (change in one place)
- ✓ Reduces errors and improves maintainability
- ✓ Makes code **self-documenting**

Example

Exchange rate changes
1020 → 1050

Magic Numbers

Many changes
(All occurrences)

Symbolic Constants

One change
(Definition only)

Summary

	Magic Numbers	Symbolic Constants
Example	1020, 1050	#define EXCHANGE_RATE 1020
Readability	Low	High
Maintainability	Difficult	Easy
Change Impact	Many places	One place



Use symbolic constants (e.g., **#define**, **const**) instead of magic numbers! → Write **clean, safe, and maintainable** C code.

Example: Symbolic Constants

```
#include <stdio.h>
#define TAX_RATE 0.2

int main( void )
{
    const int MONTHS = 12;
    int m_salary , y_salary ;

    printf ( "Enter your salary: " );
    scanf ( "%d" , & m_salary );
    y_salary = MONTHS * m_salary ;
    printf ( "Your annual salary is %d." , y_salary );
    printf ( "Tax is %f." , y_salary * TAX_RATE);

    return 0;
}
```

Summary & Common Mistakes

- Program flow: design -> write -> compile -> link -> run -> debug.
- Every program has comments, #include, and a main() function; statements end with ; and are executed sequentially.
- **printf()** outputs, **scanf()** inputs (remember & for **scanf** targets).
- Choose the right **data type** for your data, and initialize your variables - watch for overflow and floating-point rounding errors.
- Every **variable** must be declared with a type before it is used.
- Don't confuse = (assignment) with == (equality) - mixing them up is one of the most common beginner bugs.
- A missing **semicolon** or mismatched { } is the most common first compile error you'll see.
- Compiling has stages: preprocess -> compile -> assemble -> link
 - source code (.c) becomes object code, then an executable.
- Using the wrong **format specifier** in printf/scanf (e.g. %d for a float) often compiles but prints garbage.

Q & A

Lab & Quiz: right after this - see you in the practice session!